

✓ All Relationships in Java (Class Relationships) -- OPPs Concepts

Java supports **six major relationships** between classes.

1 Inheritance (IS-A Relationship)

Meaning: One class acquires properties of another.

Keyword: extends

Example

```
// Parent class
class Vehicle {
    void start() {
        System.out.println("Vehicle starts...");
    }
}

// Child class
class Car extends Vehicle { // Car IS-A Vehicle
    void drive() {
        System.out.println("Car is driving...");
    }
}

public class Main {
    public static void main(String[] args) {
        Car c = new Car();
        c.start(); // inherited from Vehicle
        c.drive(); // Car's own method
    }
}
```

Real-life

Car is a vehicle
Employee **is a** Person

🔍 How It Works

- Car extends Vehicle
- Car gets **all methods** of Vehicle
- This relation is called **IS-A** (Car **is a** Vehicle)

2 Composition (HAS-A Strong Relationship)

Meaning: One class **owns** another class. Strong lifecycle. If the container dies → contained object also dies.

Example

```
// Engine class (part of Car)
class Engine {
    void startEngine() {
        System.out.println("Engine starts...");
    }
}

// Car class (has an Engine)
class Car {

    // Strong relationship → Car owns Engine
    private Engine engine = new Engine();

    void startCar() {
        System.out.println("Car starting...");
        engine.startEngine(); // using Engine inside Car
    }
}

public class TestCar {
    public static void main(String[] args) {
        Car c = new Car();
        c.startCar();
    }
}
```

✓ Car **has-a** Engine

✓ Engine *cannot exist* without Car (strong relationship)

Why This Is Composition?

- Car **creates** the Engine object internally.
 - Engine **cannot exist** without Car.
 - When Car is destroyed → Engine is also destroyed.
-

3 Aggregation (HAS-A Weak Relationship)

Meaning: One class uses another but **lifecycle is independent**.

Example

```
import java.util.ArrayList;
import java.util.List;

// Independent class
class Student {
    String name;

    Student(String name) {
        this.name = name;
    }
}

// School class HAS students (but does NOT own them strongly)
class School {
    // Aggregation: School just holds a reference to existing students
    List<Student> students;

    School(List<Student> students) {
        this.students = students;
    }

    void showStudents() {
        System.out.println("Students in school:");
        for (Student s : students) {
            System.out.println("- " + s.name);
        }
    }
}

public class TestAggregation {
    public static void main(String[] args) {

        // Students exist independently
        Student s1 = new Student("Aniket");
        Student s2 = new Student("Rahul");
        Student s3 = new Student("Pooja");

        // Passing students to school (School does not create them)
        List<Student> list = new ArrayList<>();
        list.add(s1);
        list.add(s2);
        list.add(s3);

        School school = new School(list);
        school.showStudents();
    }
}
```

}

✓ School **has** Students, but students can exist outside school.

🔍 Why This Is Aggregation?

- Student objects are created **outside** the School.
- School only **uses references** to Student.
- Even if the School object is destroyed → Student objects still exist.
- This shows **weak HAS-A** relationship.

4 Association (General Relationship)

Meaning: Two classes interact with each other. Not owner, not parent—just interaction.

Example

```
// Driver class
class Driver {
    String name;

    Driver(String name) {
        this.name = name;
    }

    void drive() {
        System.out.println(name + " is driving the car.");
    }
}
```

```
// Car class
class Car {
    String model;
    Driver driver; // Association: Car uses Driver

    Car(String model) {
        this.model = model;
    }

    void assignDriver(Driver driver) {
        this.driver = driver;
    }

    void showDetails() {
        System.out.println("Car model: " + model);

        if (driver != null) {
            driver.drive();
        } else {
            System.out.println("No driver assigned.");
        }
    }
}
```

```

    }
}

public class TestAssociation {
    public static void main(String[] args) {
        // Driver exists independently
        Driver d = new Driver("Aniket");

        // Car exists independently
        Car car = new Car("Maruti Swift");

        // They are connected through association
        car.assignDriver(d);

        car.showDetails();
    }
}

```

✓ Driver drives a Car → general association.

Why This Is Association?

- Car and Driver exist independently.
- No lifetime dependency.
- They are simply **associated** at runtime (driver drives a car).

5 Dependency (USE Relationship)

Meaning: One class **depends** on another class temporarily.
Used in methods only.

Example

```

// Payment gateway class
class PaymentGateway {
    void processPayment() {
        System.out.println("Payment processed successfully.");
    }
}

// Order class uses PaymentGateway temporarily
class Order {

    void makePayment(PaymentGateway pg) { // Dependency
        System.out.println("Order: Initiating payment...");
        pg.processPayment();           // using pg temporarily
    }
}

```

```

public class TestDependency {
    public static void main(String[] args) {

        // Created temporarily and passed to method
        PaymentGateway gateway = new PaymentGateway();

        Order order = new Order();
        order.makePayment(gateway);    // using pg only for this call
    }
}

```

✓ Order *uses* PaymentGateway only in method → dependency.

Why This Is Dependency?

- PaymentGateway object is **not stored** inside Order.
- It is just **passed to a method** to perform an operation.
- After the method finishes, the dependency ends.

6 Realization (Interface Implementation)

Meaning: Class implements an interface.

Keyword: implements

Example

```

// Interface (only method declarations)
interface Vehicle {
    void start();    // abstract method
    void stop();
}

```

```

// Class realizes (implements) the interface
class Bike implements Vehicle {

```

```

    @Override
    public void start() {
        System.out.println("Bike starts with self-start.");
    }

```

```

    @Override
    public void stop() {
        System.out.println("Bike stops using disc brake.");
    }
}

```

```

public class TestInterface {

```

```

public static void main(String[] args) {
    Vehicle bike = new Bike(); // Upcasting

    bike.start();
    bike.stop();
}
}

```

✓ Bike **realizes** Vehicle interface.

Why This Is Realization?

- Vehicle is an **interface** → behavior contract.
- Bike **implements** it → provides the actual logic.
- Relationship is called **Realization** in UML.

7 Multiplicity (1–1, 1–Many, Many–Many)

Describes **how many** objects relate.

Examples:

- One Bank → many Accounts
- One Account → one Customer
- Many Students ↔ Many Courses

★ Summary Table

Relationship	Meaning	Keyword	Lifecycle	Example
Inheritance (IS-A)	Parent → child	extends	Parent independent	Car extends Vehicle
Composition	Strong HAS-A	none	Child dies if parent dies	Car → Engine
Aggregation	Weak HAS-A	none	Independent	School → Students
Association	General link	none	Independent	Driver → Car
Dependency	Uses temporarily	none	Independent	Order → Payment
Realization	Interface → Class	implements	Independent	Car implements Vehicle

